Block 5 – SDN & Cloud-Native Networking

S1: SDN, NFV, Overlays & Network Automation

Arquitectura de Xarxes

Universitat Pompeu Fabra

Outline

What If You Could Program Your Entire Network?

Traditional networks are configured device by device.
What if the network was as programmable as software?

Learning objectives:

1. Explain SDN architecture and why centralized control matters
2. Understand NFV and how it replaces hardware appliances
3. Describe overlay networks (VXLAN) and why they scale beyond VLANs
4. Understand how containers communicate over bridge and overlay networks
5. Explain Infrastructure as Code and why it enables network automation

Traditional Network Management

- Each device (router, switch, firewall) configured **individually**
- Configuration via CLI, **device by device**
- Vendor-specific commands and interfaces
- Changes require **manual intervention** on every device

Imagine updating 1,000 switches one by one. That's traditional networking.

Scalability Challenges

- Data centers grow from hundreds to **hundreds of thousands** of devices
- Manual configuration does **not scale**
- Network changes are **slow**: days or weeks for approval + implementation
- **Human errors** increase with complexity
- Cloud-scale infrastructure demands **automation**

The Control Plane Problem

- In traditional networks, **every device** runs its own control plane
- Each router independently computes routes (OSPF, BGP from Block 3)
- No **centralized view** of the entire network
- Difficult to implement **network-wide policies**
- Limited **programmability**: cannot easily add new features

The control plane is **distributed** by design. What if we **centralized** it?

Discussion: The Cost of Manual Networks

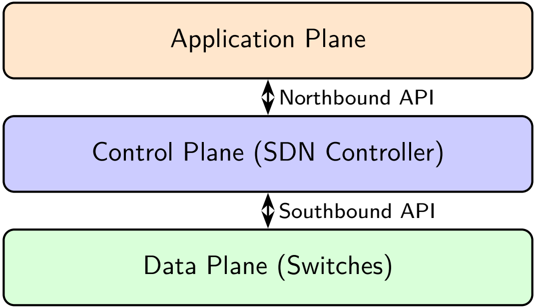
Why have traditional networks survived so long despite their scalability problems?

- Hint: think about reliability, vendor relationships, and risk aversion
- What would it take for an organization to change?

Outline

SDN – Core Idea

- **Separate** the control plane from the data plane
- **Centralize** network intelligence in a software controller
- **Program** network behavior through APIs



McKeown et al., "OpenFlow: Enabling Innovation in Campus Networks," *ACM SIGCOMM CCR*, 2008.

SDN Architecture – Three Planes

Data Plane (Infrastructure Layer):

- Physical/virtual switches that **forward packets**
- No longer make independent routing decisions
- Receive forwarding rules from the controller

Control Plane (Controller):

- Centralized software with a **global view** of the network
- Computes paths, installs forwarding rules on switches
- Single point of management for the entire network

Application Plane:

- Applications that define **network behavior** (firewall, load balancer, monitor)
- Communicate with the controller via northbound APIs

SDN Controllers

- The controller is the **brain** of an SDN network
- Maintains a real-time **topology database** of the entire network
- Supports **high availability** (clustered deployment)

Examples:

- **OpenDaylight** (Linux Foundation, Java-based, widely adopted)
- **ONOS** (Open Network Operating System, carrier-grade, telecom-focused)
- **Floodlight** (open-source, lightweight, educational)
- Proprietary: **Cisco ACI**, **VMware NSX**

OpenDaylight Project, opendaylight.org; ONOS Project, onosproject.org.

Southbound Interface – OpenFlow

- **OpenFlow** (2008, Stanford): the original SDN protocol
- Defines how the controller communicates with switches
- Controller installs **flow rules** in the switch's flow table:

Match	Action	Priority
dst IP = `10.0.1.0/24`	Forward to port 3	100
dst IP = `10.0.2.0/24`	Forward to port 5	100
src IP = `192.168.1.100`	Drop	200
* (any)	Send to controller	1

- If no rule matches → packet sent to controller for a decision

ONF, "Software-Defined Networking: The New Norm for Networks," ONF White Paper, 2012.

Northbound Interface – Representational State Transfer (REST) APIs

- Applications interact with the controller via **REST APIs**
- Example operations:
  - `GET /topology/links` → get network topology
  - `POST /flows` → add a flow rule
  - `GET /statistics/port` → query traffic statistics

Key benefit: the network becomes **programmable** like any other software system.

Any developer can write applications that control network behavior.

SDN Benefits – Summary

- **Centralized management**: one controller, global network view
- **Programmability**: automate changes via APIs
- **Agility**: deploy new policies in seconds, not days
- **Vendor independence**: OpenFlow works across vendors
- **Innovation**: researchers and developers can experiment easily

SDN is the foundation for modern cloud networking: AWS, Azure, and GCP all use SDN internally.

Discussion: SDN Trade-offs

If the SDN controller fails,
what happens to the network?

- Hint: single point of failure vs distributed control
- How do real deployments address this? (clustering, failover)

Outline

From Hardware Appliances to Software

- Traditional network functions run on **dedicated hardware**:
  - Firewall $\rightarrow$ physical appliance ($10K–$100K+)
  - Load balancer $\rightarrow$ physical appliance
  - Router $\rightarrow$ physical appliance
- Problems: **expensive**, inflexible, vendor lock-in, slow to deploy
- NFV idea: run these functions as **software on commodity servers**

ETSI, "Network Functions Virtualisation – An Introduction, Benefits, Enablers, Challenges & Call for Action," White Paper, 2012.

Virtual Network Functions (VNFs)

- A **VNF** = network function implemented as software
- Runs on VMs or containers, on standard x86 servers

Physical Appliance	**VNF Equivalent**
Hardware firewall	Virtual firewall (pfSense, iptables)
Hardware load balancer	Virtual LB (HAProxy, NGINX)
Hardware router	Virtual router (VyOS, FRRouting)
WAN optimizer	Virtual WAN optimizer
Intrusion Detection / Prevention (IDS/IPS)	Virtual IDS (Snort, Suricata)

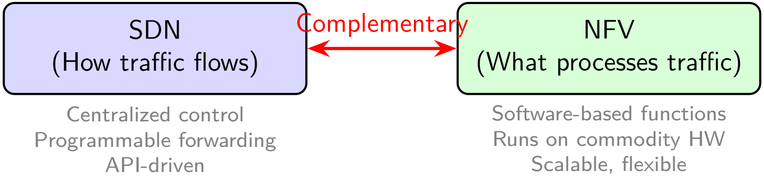
ETSI GS NFV 002, "NFV Architectural Framework," v1.2.1, 2014.

NFV + Cloud Integration

- Deploy VNFs on cloud infrastructure (IaaS) $\rightarrow$ **minutes** instead of months
- **Scale horizontally**: add more instances under load
- **Update easily**: software upgrades, no truck rolls
- **Cost reduction**: commodity servers vs specialized hardware

No hardware procurement, no shipping, no rack-and-stack.

SDN + NFV – Complementary Technologies



- SDN **steers** traffic to the right VNF (forwarding rules)
- NFV **processes** the traffic (filter, balance, encrypt, inspect)

Discussion: When Hardware Still Wins

Can you think of a scenario where a physical appliance is better than a VNF?

- Hint: think about latency, throughput, and specialized workloads
- What about hardware accelerators (FPGAs, SmartNICs)?

Outline

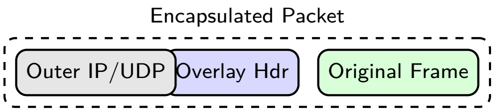
The VLAN Scalability Problem

- In Block 4 we used **VLANs** to isolate tenants on virtual networks
- VLANs use a **12-bit ID** → maximum **4,096** virtual networks
- Cloud providers host **millions** of tenants → VLANs are not enough
- Additional limitations:
  - VLANs are confined to a **single L2 domain**
  - Moving a VM to another host may require VLAN reconfiguration

We need a technology that scales beyond 4K networks and works across L3 boundaries.

Overlay Networks: Concept

- An **overlay network** is a virtual network built **on top of** the physical one
- Uses **encapsulation**: wrap virtual frames inside physical packets



- The physical network only sees the **outer headers**
- The virtual (tenant) traffic is hidden inside → **full isolation**

VXLAN: Overview

- **VXLAN** (Virtual eXtensible LAN): most widely used overlay protocol
- Encapsulates L2 frames in **UDP packets** over the physical network
- Uses a **24-bit VNI** (VXLAN Network Identifier)
  - 2^{24} = **16 million** virtual networks (vs 4,096 VLANs)
- Endpoints: **VTEPs** (VXLAN Tunnel Endpoints)
  - Encapsulate at source, decapsulate at destination

Source: IETF RFC 7348, "Virtual eXtensible Local Area Network (VXLAN)," 2014.

VXLAN: How It Works

- VM A sends a frame → VTEP 1 wraps it in UDP with a VNI
- Travels over the physical network as a regular UDP packet
- VTEP 2 strips outer headers, delivers original frame to VM B
- VMs on the **same VNI** form an isolated L2 domain

Overlay Benefits for Scalability

- **16 million virtual networks** (vs 4,096 VLANs)
- Physical network does not need to know about tenants
- VMs can **migrate** across hosts without reconfiguring the underlay
- Decouples **virtual topology** from physical topology

Overlays are the foundation of cloud networking: every virtual network (VPC) you create runs on top of VXLAN or similar protocols.

Source: IETF RFC 8926, "Geneve: Generic Network Virtualization Encapsulation," 2020.

Discussion: Overlay Networks

Why can the physical network remain simple
if we use overlay networks?
What are the trade-offs of encapsulation?

Hints: think about overhead (extra headers), MTU implications, and troubleshooting complexity.

Outline

Container Networking: Starting Point

- In Block 4 and Lab 3 we covered what containers are and how Docker works
- Here we focus on the **networking layer**: how containers connect to each other and to the outside world
- Every container gets its own **network namespace**: isolated IP stack, interfaces, routing table
- The container runtime (Docker) wires containers together using **virtual bridges** and **veth pairs**

Container Network Models

Bridge network (default):

- Containers on the same host connected via a **virtual bridge** (`docker0`)
- Each container gets a **veth pair** (virtual Ethernet interface)
- **NAT** for external traffic; containers share host's IP

Host network:

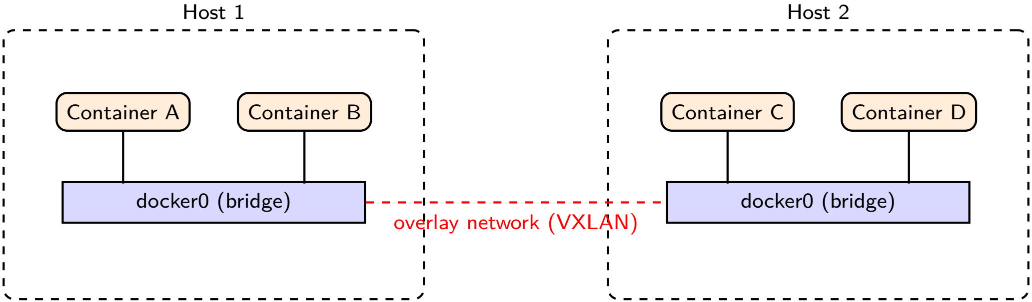
- Container uses the **host's network stack directly**
- No network isolation, but **no NAT overhead**
- Use for performance-critical applications

Overlay network:

- Containers on **different hosts** connected via VXLAN overlay
- Same overlay concept we just covered, now applied to containers
- Enables **multi-host** container deployments

Docker, Inc., "Docker Networking Documentation," docs.docker.com/network.

Container Networking – Visual



- Within a host: containers use a **bridge** network
- Across hosts: an **overlay** connects the bridges (VXLAN from the previous section)

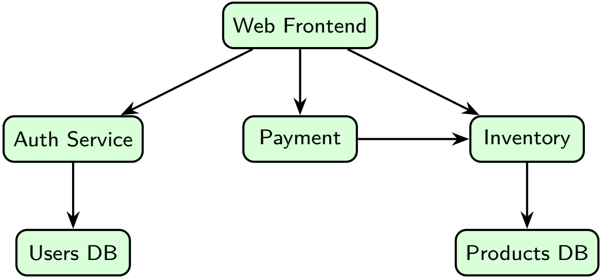
Microservices – From Monolith to Distributed

- **Monolithic** app: one big process, one deployment, one codebase
- **Microservices**: app split into small, independent services
  - Each service runs in its **own container**
  - Services communicate over the **network** (HTTP/REST, gRPC Remote Procedure Call)

Network implications:

- Hundreds of containers talking to each other
- Need **service discovery**: "where is the payment service?"
- Need **load balancing**: distribute requests across replicas
- Need **observability**: trace requests across services

Microservices – Network Challenges



- Every arrow = **network call** → latency and failure risk
- If Inventory is down → Payment and Web are affected (**cascading failure**)
- Solutions: retries, timeouts, circuit breakers, service meshes

Discussion: Containers vs VMs

If containers are faster and lighter than VMs,
why do companies still use VMs?

- Hint: security isolation, legacy applications, compliance requirements
- In practice, containers often run inside VMs

Outline

Infrastructure as Code (IaC)

- **IaC**: manage infrastructure through **code files**, not manual clicks
- Describe desired state in a configuration file → tools apply it automatically

Key benefits:

- **Reproducibility**: same config → same infrastructure, every time
- **Version control**: track changes in Git, review, rollback
- **Automation**: no manual steps → fewer human errors
- **Speed**: deploy entire environments in minutes

Morris, *Infrastructure as Code*, 2nd ed., O'Reilly, 2021.

IaC Example – Terraform (Conceptual)

```
# Define a VPC
resource "aws_vpc" "main" {
  cidr_block = "10.0.0.0/16"
}

# Define a subnet
resource "aws_subnet" "public" {
  vpc_id     = aws_vpc.main.id
  cidr_block = "10.0.1.0/24"
}

# Define a security group
resource "aws_security_group" "web" {
  vpc_id = aws_vpc.main.id
  ingress {
    from_port = 80
```

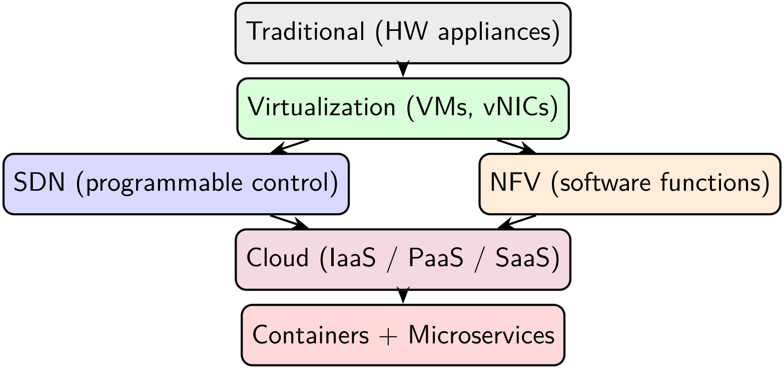
Discussion: IaC Risks

If infrastructure is defined as code,
what happens when someone pushes a bug?

- Hint: code review, staging environments, automated testing
- How is this similar to software development best practices?

Outline

The Full Picture – Where Everything Fits



Each layer builds on the previous one. This is the **evolution of networking**.

References

1. McKeown et al., "OpenFlow: Enabling Innovation in Campus Networks," *ACM SIGCOMM CCR*, vol. 38, no. 2, 2008.
2. ONF, "Software-Defined Networking: The New Norm for Networks," ONF White Paper, 2012.
3. OpenDaylight Project, opendaylight.org.
4. ONOS Project, onosproject.org.
5. ETSI, "Network Functions Virtualisation," White Paper, 2012.
6. ETSI GS NFV 002, "NFV Architectural Framework," v1.2.1, 2014.
7. Docker, Inc., "Docker Networking Documentation," docs.docker.com/network.
8. Morris, *Infrastructure as Code*, 2nd ed., O'Reilly, 2021.
9. HashiCorp, "Terraform Documentation," terraform.io/docs.
10. Goransson & Black, *Software Defined Networks*, 2nd ed., Morgan Kaufmann, 2017.
11. Kurose & Ross, *Computer Networking: A Top-Down Approach*, 8th ed., Pearson, 2021.
12. IETF RFC 7348, "Virtual eXtensible Local Area Network (VXLAN)," 2014.
13. IETF RFC 8926, "Geneve: Generic Network Virtualization Encapsulation," 2020.

Key Takeaways

1. Traditional networks are **manual, rigid, and do not scale**
2. **SDN** separates control from data plane: centralized, programmable, API-driven
3. **NFV** replaces hardware appliances with software: flexible, fast to deploy, cheap
4. SDN and NFV are **complementary**: SDN steers traffic, NFV processes it
5. **VXLAN** overlays scale isolation to 16 million networks (vs 4,096 VLANs)
6. Containers use **bridge networks** on a single host and **overlay networks** across hosts
7. **Microservices** = many containers communicating over the network, requiring service discovery and resilience
8. **IaC** automates infrastructure: reproducible, version-controlled, auditable

Discussion

A company runs 200 microservices in containers.
One Friday at 5 PM, a manual network change
breaks communication for 50 of them.
How could SDN and IaC have prevented this?

Hints: centralized control, automated testing, instant rollback, reproducibility.